```
-- ============================================================
-- SQL QUESTION & ANSWER REFERENCE (QUESTIONS 1 TO 60)
-- ============================================================


-- 1. Create a table Users with a primary key user_id and a unique email. What is the difference between them?
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    email VARCHAR(255) UNIQUE
);
-- Difference: PRIMARY KEY uniquely identifies each row and CANNOT contain NULLs (only 1 per table).
-- UNIQUE ensures all values are distinct, but CAN contain NULLs (multiple UNIQUE constraints allowed per table).


-- 2. Build a report with one column labelled by type: all customer names and all supplier names together
SELECT name AS type FROM Customers
UNION ALL
SELECT name AS type FROM Suppliers;


-- 3. Find employees who have the highest salary in the company.
SELECT *
FROM Employees
WHERE salary = (SELECT MAX(salary) FROM Employees);


-- 4. Find orders with an amount higher than the overall average order amount.
SELECT *
FROM Orders
WHERE amount > (SELECT AVG(amount) FROM Orders);


-- 5. Show customers whose total order amount is above 1000, sorted from highest to lowest.
SELECT customer_id, SUM(amount) AS total_amount
FROM Orders
GROUP BY customer_id
HAVING SUM(amount) > 1000
ORDER BY total_amount DESC;


-- 6. Count employees for each combination of department and city.
SELECT department, city, COUNT(*) AS employee_count
FROM Employees
GROUP BY department, city;


-- 7. Sort employees by department ascending, then by salary descending within each department.
SELECT *
FROM Employees
ORDER BY department ASC, salary DESC;


-- 8. Select employees who are NOT (in Baku or earning less than 1000).
SELECT *
```

```
FROM Employees
WHERE NOT (city = 'Baku' OR salary < 1000);


-- 9. Copy all employees from Baku into the table Employees_Archive (same structure).
INSERT INTO Employees_Archive
SELECT *
FROM Employees
WHERE city = 'Baku';


-- 10. Delete duplicate employees (same first_name and last_name) keeping the one with the smallest emp_id.
DELETE FROM Employees
WHERE emp_id NOT IN (
    SELECT MIN(emp_id)
    FROM Employees
    GROUP BY first_name, last_name
);


-- 11. Set the salary to the company average for all employees whose salary is NULL.
UPDATE Employees
SET salary = (SELECT AVG(salary) FROM Employees)
WHERE salary IS NULL;


-- 12. Find the average of the department salary totals (subquery in FROM).
SELECT AVG(dept_total) AS avg_department_salary_total
FROM (
    SELECT department, SUM(salary) AS dept_total
    FROM Employees
    GROUP BY department
) AS dept_salaries;


-- 13. Find customers who have never placed an order.
SELECT *
FROM Customers
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM Orders
);


-- 14. Find the second highest salary.
SELECT MAX(salary) AS second_highest_salary
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);


-- 15. Find employees who earn more than the average salary of their own department (correlated subquery).
SELECT *
FROM Employees e1
WHERE salary > (
```

```
    SELECT AVG(salary)
    FROM Employees e2
    WHERE e2.department = e1.department
);


-- 16. Find duplicate last names in Employees.
SELECT last_name, COUNT(*) AS count
FROM Employees
GROUP BY last_name
HAVING COUNT(*) > 1;


-- 17. Get all cities from Customers and Suppliers (no duplicates), sorted alphabetically.
SELECT city FROM Customers
UNION
SELECT city FROM Suppliers
ORDER BY city ASC;


-- 18. Show the average salary per department, only for employees hired after 2020, and only
departments with an average above 3000.
SELECT department, AVG(salary) AS avg_salary
FROM Employees
WHERE hire_date > '2020-12-31'
GROUP BY department
HAVING AVG(salary) > 3000;


-- 19. Create Orders with a foreign key that automatically deletes orders when the customer is deleted.
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    amount DECIMAL(10, 2),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE CASCADE
);


-- 20. Create a table OrderItems where the primary key is a combination of order_id and product_id.
CREATE TABLE OrderItems (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id)
);


-- 21. Can you use ORDER BY inside a subquery?
-- Answer: Yes, but it only makes functional sense when paired with LIMIT (or TOP/ROWNUM). Without
LIMIT, SQL discards subquery ordering.
SELECT *
FROM (
    SELECT *
    FROM Employees
    ORDER BY salary DESC
    LIMIT 5
```

```
) AS top_salaries;


-- 22. What is the result of SELECT 1 UNION SELECT 1 UNION ALL SELECT 1?
-- Answer: Returns 2 rows containing '1'.
-- Explanation: 'SELECT 1 UNION SELECT 1' produces one '1' (removes duplicates). 'UNION ALL
SELECT 1' keeps duplicates, adding a second '1'.
SELECT 1
UNION
SELECT 1
UNION ALL
SELECT 1;


-- 23. What does SELECT DISTINCT do and how is it different from GROUP BY?
-- SELECT DISTINCT removes duplicate rows across selected columns:
SELECT DISTINCT department FROM Employees;
-- GROUP BY groups rows to perform calculations (SUM, COUNT, AVG) using aggregate functions:
SELECT department, COUNT(*) FROM Employees GROUP BY department;


-- 24. Why does WHERE column = NULL return nothing, and how do you fix it?
-- Answer: NULL represents an unknown value. Comparisons like '= NULL' result in UNKNOWN (false).
-- Fix: Use 'IS NULL' or 'IS NOT NULL'.
SELECT * FROM Employees WHERE salary IS NULL;


-- 25. What do ANY and ALL mean with subqueries?
-- ANY: Returns TRUE if the comparison matches AT LEAST ONE value in the subquery result.
SELECT * FROM Employees WHERE salary > ANY (SELECT salary FROM Employees WHERE
department = 'IT');
-- ALL: Returns TRUE only if the comparison matches ALL values in the subquery result.
SELECT * FROM Employees WHERE salary > ALL (SELECT salary FROM Employees WHERE
department = 'IT');


-- 26. How do you delete duplicate rows while keeping one copy?
DELETE FROM Employees
WHERE emp_id NOT IN (
        SELECT MIN(emp_id)
        FROM Employees
        GROUP BY first_name, last_name
);


-- 27. What is INSERT ... SELECT? (INSERT in SELECT)
-- Answer: It copies data directly from a SELECT query into an existing table.
INSERT INTO Employees_Archive
SELECT * FROM Employees WHERE city = 'Baku';


-- 28. How do you update a column using a value from a subquery?
UPDATE Employees
SET salary = (SELECT AVG(salary) FROM Employees)
WHERE salary IS NULL;
```

-- 29. How does ORDER BY work with multiple columns, and can it use aliases?
-- Answer: It sorts by the first column first; tied values are then sorted by the second column. Yes, ORDER BY can use aliases.
SELECT first_name, salary AS emp_salary
FROM Employees
ORDER BY department ASC, emp_salary DESC;

-- 30. What is the difference between COUNT(*), COUNT(column), and COUNT(DISTINCT column)?
SELECT COUNT(*) FROM Employees; -- Counts ALL rows (including NULLs)
SELECT COUNT(salary) FROM Employees; -- Counts non-NULL values in 'salary'
SELECT COUNT(DISTINCT department) FROM Employees; -- Counts UNIQUE non-NULL departments

-- 31. What is the precedence of NOT, AND, and OR? Evaluate A OR B AND C.
-- Answer: Order is 1. NOT, 2. AND, 3. OR.
-- Evaluation: 'A OR B AND C' is evaluated as 'A OR (B AND C)' because AND executes before OR.
*SELECT * FROM Employees WHERE department = 'IT' OR city = 'Baku' AND salary > 1000;*

-- 32. Why can NOT IN return no rows when the subquery contains NULL?
-- Answer: 'x NOT IN (1, NULL)' translates to 'x != 1 AND x != NULL'. Since 'x != NULL' is UNKNOWN, the entire WHERE clause fails.
-- Fix: Filter out NULLs in the subquery or use NOT EXISTS.
*SELECT * FROM Customers WHERE customer_id NOT IN (SELECT customer_id FROM Orders WHERE customer_id IS NOT NULL);*

-- 33. Compare DELETE, TRUNCATE, and DROP.
DELETE FROM Employees WHERE city = 'Baku'; -- DML: Deletes specific rows, can use WHERE, can be rolled back.
TRUNCATE TABLE Employees; -- DDL: Removes ALL rows fast, resets auto-increment, cannot use WHERE.
DROP TABLE Employees; -- DDL: Deletes table structure and all data permanently.

-- 34. What is the logical order of execution of a SQL query?
-- Answer: 1. FROM/JOIN -> 2. WHERE -> 3. GROUP BY -> 4. HAVING -> 5. SELECT -> 6. DISTINCT -> 7. ORDER BY -> 8. LIMIT

-- 35. Why can't you use aggregate functions in WHERE? What should you use instead?
-- Answer: WHERE filters individual rows BEFORE grouping and aggregation occur.
-- Fix: Use the HAVING clause to filter aggregated results.
SELECT department, AVG(salary) FROM Employees GROUP BY department HAVING AVG(salary) > 2000;

-- 36. Where do you put ORDER BY when using UNION?
-- Answer: At the very end of the final query statement.
SELECT name FROM Customers
UNION
SELECT name FROM Suppliers
ORDER BY name ASC;

-- 37. Table A has values (1, 2, 2) and table B has (2, 3). What do UNION and UNION ALL return?
-- UNION (removes duplicates): Returns 1, 2, 3
-- UNION ALL (keeps duplicates): Returns 1, 2, 2, 2, 3

-- 38. What conditions must two SELECT queries meet to be combined with UNION?
-- Answer:
-- 1. Must return the exact same number of columns.
-- 2. Corresponding columns must have matching/compatible data types.
-- 3. Columns must be placed in the exact same order in both queries.

-- 39. What happens when you insert a child row whose foreign key value does not exist in the parent table?
-- Answer: SQL throws a Foreign Key Constraint Violation error and cancels the operation.

-- 40. What is the difference between a primary key and a unique key?
-- PRIMARY KEY: Identifies rows uniquely, NO NULLs allowed, maximum 1 per table.
-- UNIQUE KEY: Prevents duplicate values, NULLs are allowed, multiple allowed per table.

-- 41. Which aggregate function counts the number of rows?
SELECT COUNT(*) FROM Employees;

-- 42. What is a subquery?
-- Answer: A query embedded inside another SQL statement (e.g., inside WHERE, FROM, or SELECT).
*SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees);*

-- 43. What is the WHERE clause used for?
-- Answer: Filters individual rows based on specific conditions BEFORE grouping occurs.
*SELECT * FROM Employees WHERE city = 'Baku';*

-- 44. What is the HAVING clause used for?
-- Answer: Filters aggregated group data AFTER the GROUP BY clause has executed.
SELECT department, AVG(salary) FROM Employees GROUP BY department HAVING AVG(salary) > 2000;

-- 45. What is the default sort order of ORDER BY?
-- Answer: Ascending order (ASC).
*SELECT * FROM Employees ORDER BY salary;*

-- 46. What does GROUP BY do?
-- Answer: Groups rows with matching values across specified columns into single summary rows.
SELECT department, COUNT(*) FROM Employees GROUP BY department;

-- 47. Is TRUNCATE a DDL or DML command?
-- Answer: TRUNCATE is a DDL (Data Definition Language) command.

-- 48. Name one difference between DELETE and TRUNCATE.
-- Answer: TRUNCATE removes all rows immediately without logging individual row deletions and resets auto-increment IDs; DELETE allows a WHERE clause to target specific rows.

-- 49. What does TRUNCATE do?
-- Answer: Removes all records from a table quickly while maintaining its structure.
TRUNCATE TABLE Employees;

-- 50. What does DELETE do?
-- Answer: Removes specific rows (or all rows if no WHERE condition is specified) from a table.
DELETE FROM Employees WHERE emp_id = 5;

-- 51. What happens if you run UPDATE without a WHERE clause?
-- Answer: It updates the specified column for EVERY single row in the table.
UPDATE Employees SET salary = 3000;

-- 52. What does UNION ALL do?
-- Answer: Combines results from multiple SELECT queries while preserving all duplicate rows.
SELECT name FROM Customers UNION ALL SELECT name FROM Suppliers;

-- 53. What does UNION do?
-- Answer: Combines results from multiple SELECT queries and automatically strips out duplicate rows.
SELECT name FROM Customers UNION SELECT name FROM Suppliers;

-- 54. What does the SELECT statement do?
-- Answer: Retrieves and displays rows and columns from database tables.
SELECT first_name, salary FROM Employees;

-- 55. Can a foreign key column contain duplicate values?
-- Answer: Yes. Multiple records in a child table can refer to the same primary key in a parent table (e.g., many employees in 1 department).

-- 56. Can a foreign key contain NULL values?
-- Answer: Yes, unless explicitly defined with NOT NULL. A NULL foreign key means the record is unlinked.

-- 57. What is a foreign key?
-- Answer: A column that references a PRIMARY KEY in another table to enforce relational integrity between tables.
```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

-- 58. How many primary keys can a table have?
-- Answer: Exactly ONE primary key per table (which can be a single column or a composite key spanning multiple columns).

-- 59. Can a primary key contain NULL values?
-- Answer: No. Primary keys MUST contain distinct, non-NULL values for every row.

-- 60. What is a primary key?
-- Answer: A column (or set of columns) that uniquely identifies each individual row in a database table.

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50)
);
```